

AI ASSISTED CODING

ASSIGNMENT: 19.2

NAME: KADARLA HASINI

HT.NO:2403A52062

BATCH: 01

TASK 1

TASK DESCRIPTION:

You are part of a multinational development team. The backend is written in Java, but a new module requires a Python implementation of the same algorithm for integration with a data science pipeline.

- Task 1: Use AI-assisted coding to translate a given Java bubble sort program into Python. Verify that the translated code works correctly.**
- Task 2: Introduce errors in the Python version to check if the input list is empty or contains non-numeric values**

PROMPT

"Translate a Java bubble sort algorithm into Python, verify its correctness with a test case, and implement error handling for empty or non-numeric input lists."

CODE & OUTPUT

```
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n - 1):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater
            # than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

print("Bubble sort function defined.")
```

•• Bubble sort function defined.

```
test_list = [64, 34, 25, 12, 22, 11, 90]
print(f"Original list: {test_list}")

sorted_list = bubble_sort(test_list)
print(f"Sorted list: {sorted_list}")
```

... Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]

CODE EXPLANATION

The provided code first defines a Python `bubble_sort` function, which sorts a list by repeatedly stepping through the list, comparing adjacent elements, and swapping them if they are in the wrong order. After defining the function, a `test_list` is created. The `bubble_sort` function is then called with this list, and both the original and sorted lists are printed to verify the algorithm's correct functionality.

TASK 2

TASK DESCRIPTION

A company's legacy codebases stores and processes files in C++, but the analytics team needs an equivalent program in JavaScript (Node.js) for integration with a web dashboard.

- Task 1: Translate a given C++ file read-and-write program into JavaScript using AI assistance. Ensure the script reads a text file and writes processed output to a new file.
- Task 2: Add error handling in the JavaScript version to gracefully handle missing files or permission errors

PROMPT

"Translate a C++ file read-and-write program into JavaScript (Node.js) that reads a text file and writes processed output to a new file, including error handling for missing files or permission issues."

CODE & OUTPUT

```
import subprocess
import os

# The Node.js code provided in the previous step, now as a Python string.
nodejs_code = """
const fs = require('fs').promises;

async function processFile(inputFilePath, outputFilePath) {
  try {
    // 3. Read the content of the input file
    const content = await fs.readFile(inputFilePath, 'utf8');
    console.log(`Successfully read file: ${inputFilePath}`);

    // 4. Apply a simple processing step (e.g., convert to uppercase)
    const processedContent = content.toUpperCase();
    console.log("Content processed (converted to uppercase).");

    // 5. Write the processed content to the output file
    await fs.writeFile(outputFilePath, processedContent, 'utf8');
    console.log(`Successfully wrote processed content to: ${outputFilePath}`);

    // 6. Indicate successful processing
    console.log("File processed successfully.");
  } catch (error) {
    console.error(`Error processing file: ${error.message}`);
  }
}
```

```

# Define the path for the temporary Node.js script file
script_path = "temp_nodejs_script.js"

# Write the Node.js code to the temporary file
with open(script_path, "w") as f:
    f.write(nodejs_code)

print(f"Node.js code saved to {script_path}")

# Execute the Node.js script using the 'node' interpreter
try:
    # Assuming 'node' is available in the environment's PATH
    result = subprocess.run(["node", script_path], capture_output=True, text=True, check=True, encoding='utf-8')
    print("\nNode.js script output:\n")
    print(result.stdout)
    if result.stderr:
        print("\nNode.js script errors (if any):\n")
        print(result.stderr)
except FileNotFoundError:
    print("Error: 'node' command not found. Please ensure Node.js is installed and in your PATH.")
except subprocess.CalledProcessError as e:
    print(f"\nError running Node.js script: {e}")
    print(f"stdout: {e.stdout}")
    print(f"stderr: {e.stderr}")
finally:
    # Clean up the temporary script file
    if os.path.exists(script_path):
        os.remove(script_path)
    print(f"Cleaned up temporary script file: {script_path}")

```

```

Node.js code saved to temp_nodejs_script.js

Node.js script output:

Executing Node.js script...
Created sample input file: input.txt with content: "Hello, world! This is a test file for Node.js file processing."
Successfully read file: input.txt
Content processed (converted to uppercase).
Successfully wrote processed content to: output.txt
File processed successfully.

Content of output.txt:
HELLO, WORLD! THIS IS A TEST FILE FOR NODE.JS FILE PROCESSING.

Cleaned up temporary script file: temp_nodejs_script.js

```

CODE EXPLANATION

The Python script dynamically creates a temporary Node.js file that reads content from input.txt, converts it to uppercase, and writes the processed output to output.txt. This Node.js code, which includes try-catch blocks for error handling, is then executed via Python's subprocess module, ensuring cross-language functionality and proper cleanup of the temporary file.

TASK 3

TASK DESCRIPTION

Your team developed a prototype in Python to fetch weather data from an API, but the production environment only supports Java.

- Task 1: Translate the Python script (that makes an API call and prints the response) into Java using AI-assisted coding. Ensure equivalent functionality.
- Task 2: Add proper error handling in the Java version for cases such as invalid API key, request timeout, or no internet connection.

PROMPT

"Translate a given Python script that makes an API call and prints the response into an equivalent Java program, using AI assistance."

CODE & OUTPUT

```

java
import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import org.json.JSONObject; // Requires org.json library

public class ApiCallerWithRobustErrorHandling {

    private static final String API_URL = "https://jsonplaceholder.typicode.com/posts/1";
    private static final Duration TIMEOUT_DURATION = Duration.ofSeconds(10); // Set a request timeout

    public static void main(String[] args) {
        System.out.println("Making a GET request to: " + API_URL);

        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(TIMEOUT_DURATION) // Connection timeout
            .build();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(API_URL))
            .timeout(TIMEOUT_DURATION) // Request timeout
            .GET()
            .build();

```

Toggle Gemini

```

try {
    HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());

    int statusCode = response.statusCode();
    if (statusCode == 200) {
        System.out.println("API Response (JSON):");
        try {
            JSONObject jsonResponse = new JSONObject(response.body());
            System.out.println(jsonResponse.toString(4)); // Pretty print with indent of 4
        } catch (org.json.JSONException jsonEx) {
            System.err.println("Error parsing JSON response: " + jsonEx.getMessage());
            System.err.println("Raw response body: " + response.body());
        }
    } else if (statusCode >= 400 && statusCode < 500) {
        System.err.println("Client Error: HTTP Status Code " + statusCode);
        System.err.println("Response Body: " + response.body());
        // Specific handling for 401 Unauthorized, 403 Forbidden, 404 Not Found, etc.
    } else if (statusCode >= 500 && statusCode < 600) {
        System.err.println("Server Error: HTTP Status Code " + statusCode);
        System.err.println("Response Body: " + response.body());
    } else {
        System.err.println("Unexpected HTTP Status Code " + statusCode);
        System.err.println("Response Body: " + response.body());
    }
}

```

Toggle Gemini

```

    } else {
        System.err.println("Unexpected HTTP Status Code " + statusCode);
        System.err.println("Response Body: " + response.body());
    }
} catch (IOException e) {
    // Handles connection issues, unknown host, socket timeout, etc.
    System.err.println("Connection or I/O error occurred: " + e.getMessage());
    System.err.println("Please check your internet connection or the API URL.");
    e.printStackTrace();
} catch (InterruptedException e) {
    // Handles if the thread waiting for the response is interrupted
    System.err.println("Request interrupted: " + e.getMessage());
    Thread.currentThread().interrupt(); // Restore the interrupted status
    e.printStackTrace();
} catch (Exception e) {
    // Catches any other unexpected errors
    System.err.println("An unexpected error occurred during API call: " + e.getMessage());
    e.printStackTrace();
}

System.out.println("Java API call script executed.");

```

```

    } else if (statusCode >= 500 && statusCode < 600) {
        System.err.println("Server Error: HTTP Status Code " + statusCode);
        System.err.println("Response Body: " + response.body());
    } else {
        System.err.println("Unexpected HTTP Status Code " + statusCode);
        System.err.println("Response Body: " + response.body());
    }
} catch (IOException e) {
    // Handles connection issues, unknown host, socket timeout, etc.
    System.err.println("Connection or I/O error occurred: " + e.getMessage());
    System.err.println("Please check your internet connection or the API URL.");
    e.printStackTrace();
} catch (InterruptedException e) {
    // Handles if the thread waiting for the response is interrupted
    System.err.println("Request interrupted: " + e.getMessage());
    Thread.currentThread().interrupt(); // Restore the interrupted status
    e.printStackTrace();
} catch (Exception e) {
    // Catches any other unexpected errors
    System.err.println("An unexpected error occurred during API call: " + e.getMessage());
    e.printStackTrace();
}

System.out.println("Java API call script executed.");

```

CODE EXPLANATION

The Python script uses requests to make a GET API call, fetching and printing a JSON response, with error handling for network

issues. Its Java equivalent, generated by AI, utilizes HttpClient for the GET request and org.json to parse the response. It also incorporates try-catch blocks for IOException, InterruptedException, and checks the HTTP status code, ensuring similar functionality and robustness.